Three Products Built with AI Coding Agents

What the work looks like when it ships

Mohamed Yaakoubi

Sfax, Tunisia · amirrak8@gmail.com · +216 54711524

linkedin.com/in/yaakoubi-mohamed · github.com/mohamedyaakoubi · mohamedyaakoubi.com

I build products. I design systems and direct AI coding agents to build them, and I do not write the implementation myself. The three products here were specified, built, tested and shipped that way. Two are in the hands of users; the third is finishing its last round before release.

The pages that follow are mostly screenshots, because a product built this way should be judged on what it does, not on how it was described.

Product	What it does	Status
Himaya	Wildfire reporting and volunteer coordination for Tunisia. Android app in four languages, plus a moderation console.	Feature-complete, in final pre-release testing
SheetDiff	Compares two versions of a spreadsheet and says what happened to every row.	Live on the Google Workspace Marketplace
Career Portal	Hiring platform for a company placing international workers with European employers.	Deployed

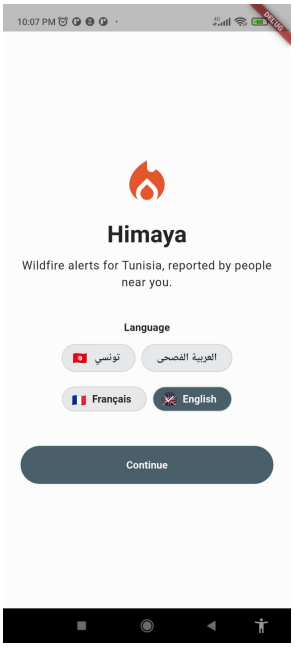
How the work divides. The agents write the implementation and propose most of the architecture. I decide what gets built, set the constraints they work inside, supply the domain knowledge they have no access to, and test the result against real conditions rather than against the description of it. Where something below was the agent's design rather than mine, it says so.

PRODUCT 1

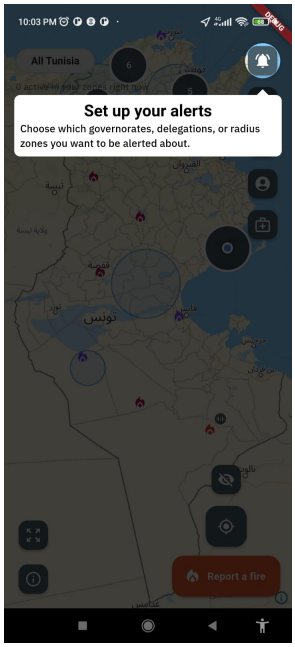
Himaya

Wildfire reporting and volunteer coordination, Tunisia

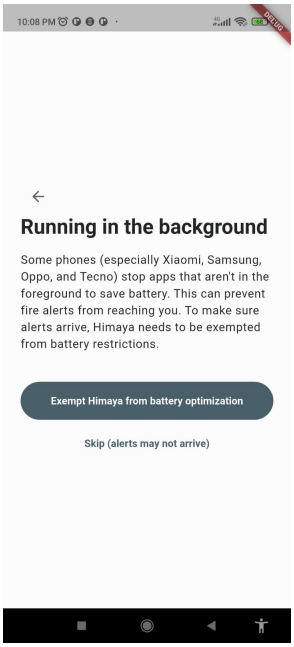
Tunisia's fire season outruns the official way of reporting a fire. Someone sees smoke on a rural road and has no fast way to reach the people close enough to act, so word travels by phone call and social media while the fire grows. Himaya turns a phone into the reporting channel: a report takes under a minute, and everyone who follows that area is alerted within seconds. It runs on Android in Tunisian dialect, Modern Standard Arabic, French and English, with a web console for moderators.



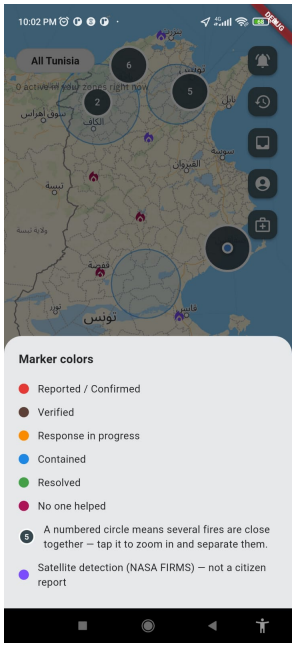
Four languages, chosen before anything else. Tunisian dialect is the default, not an afterthought.



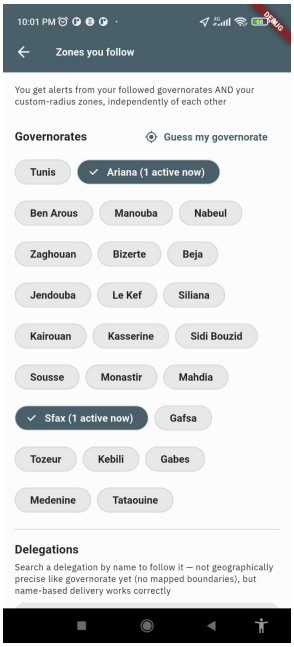
Onboarding continues inside the app, pointing at the thing it is describing.



Xiaomi, Samsung, Oppo and Tecno phones kill background apps. Setup fixes that.



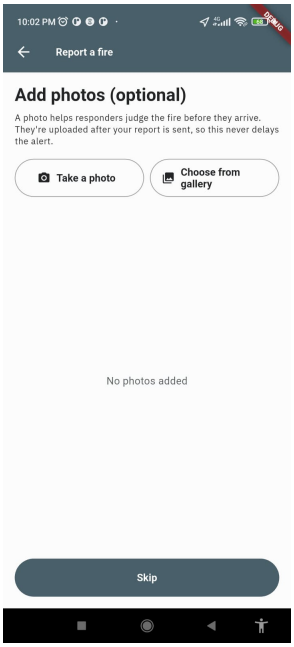
Citizen reports and satellite readings share one map, in different colours.



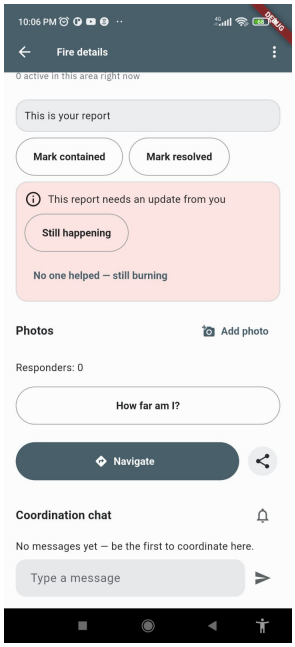
Follow whole governorates, a custom radius, or both. Delegation search states its own limitation.



Satellite alerts say they are automatic and unconfirmed, and ask the reader to verify.



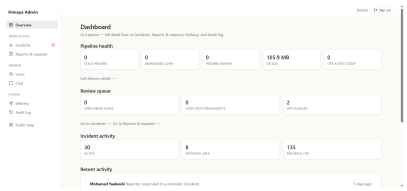
Photos are offered early but upload after the report is sent, so they never delay the alert.



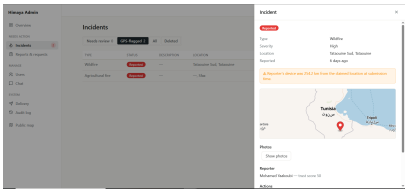
Inside a report: distance, navigation, status updates and a coordination thread.

The moderation console

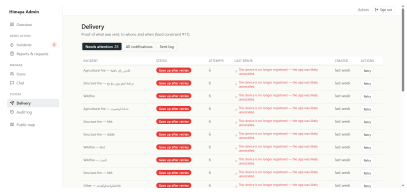
Someone has to answer for a life-safety system when it misbehaves, so the staff side is built around evidence rather than buttons.



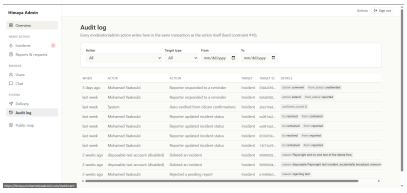
Queue depth, review backlog, reporting volume and database size on one screen.



A report filed 254 km from where the reporter's phone actually was, flagged with the reason spelled out.



Every alert, who it went to, how many attempts, and why the last one failed.



Every moderator action recorded with what changed, by whom, and when.

The decisions that shaped it

- **Reports go out first and get moderated afterwards.** Holding a fire report for review before sending it would defeat the product. Only low-scoring reports reach a moderator, and a report becomes verified when a second person independently confirms it.
- **The person who reported the fire is the one who closes it.** They are the one who can see whether it is contained. Moderators handle abuse, not outcomes.
- **Unanswered reports get their own status and colour.** When nobody responds, that is recorded rather than left to expire quietly, because it is the number the project needs to face.
- **The machine never gets the last word.** Satellite detections are labelled as automatic and unconfirmed, and ask a human to check them.
- **A report that could not have been seen gets flagged.** If the reporter's phone was hundreds of kilometres from the fire they described, a moderator is told so, with the distance, rather than being asked to guess. Reporters carry a trust score that moves with their record.
- **Delivery is provable.** A life-safety alert that silently failed is worse than none, so every send is recorded with its attempts and the reason it failed, in words a person can act on, and can be retried by hand.
- **Reporting works with no signal.** Rural Tunisia is exactly where fires start and coverage ends. A report made without a connection is held on the phone and sent by itself once signal returns, so the person who found the fire never has to remember to try again.
- **Show who else is around.** Each area displays how many people are currently active in it, so a volunteer heading out knows whether anyone else has seen the report.
- **The same fire should not arrive twice.** Satellite passes re-detect the same hotspot repeatedly. Incoming detections are matched against what is already on the map so one fire stays one entry, instead of the map filling with duplicates nobody trusts.
- **What you have already seen stays visible without signal.** Reports and the areas you follow are kept on the device, so opening the app in a dead zone still shows the last known picture rather than an empty screen and an error.

- **Limits drawn from behaviour.** More than three reports from one person inside an hour is not a citizen doing their civic duty.
- **A practice mode**, so a first-time user can rehearse without putting a false fire in front of the country.
- **Crash reports carry no one's personal details, checked twice.** Personal data is stripped at the source before a crash report leaves the phone, and a second, independent pass removes anything that could still have slipped through, such as an email address embedded in a web address. The same instinct, that one safeguard is not enough on its own, is behind every privacy-sensitive part of the app.

Designed by the agent: the database access model, the guarantee that a report and its alert are stored in one step, notification targeting, and map clustering. I set the requirement and judged the behaviour; the mechanism is theirs.

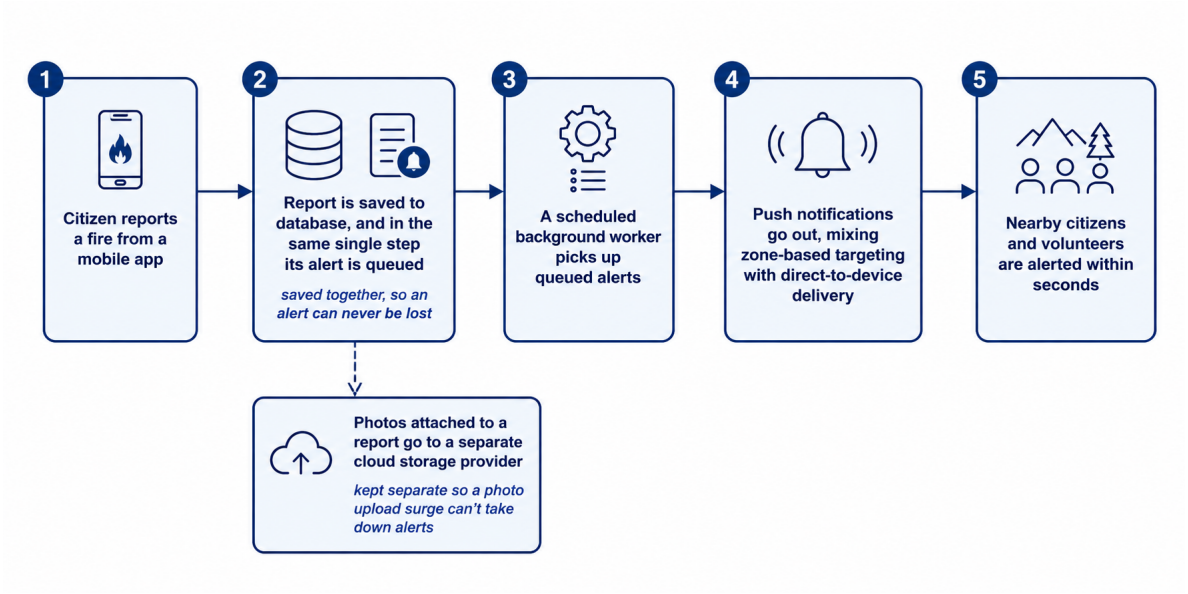


Figure 1: From report to alert. Step 2 carries the guarantee: the report and its alert are written together, so a failed network call cannot leave a fire reported and nobody told.

PRODUCT 2

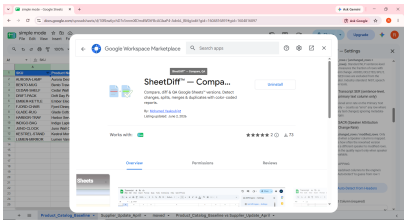
SheetDiff

Spreadsheet comparison, sold on the Google Workspace Marketplace

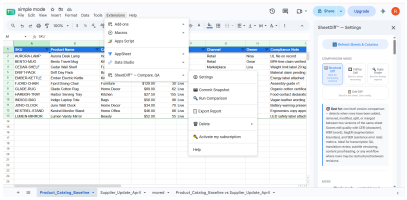
---

I was doing transcription QA: correcting other people's work, then trying to explain the corrections by sending two screenshots with arrows drawn on them. SheetDiff compares two versions of a spreadsheet and labels every row as unchanged, modified, moved, split, merged, added or deleted, so a correction explains itself. It installs into Google Sheets and sells as a subscription.

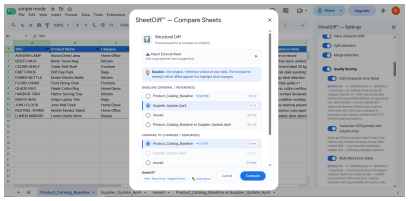
Google Workspace Marketplace listing · mohamedyaakoubi.com/en/sheetdiff



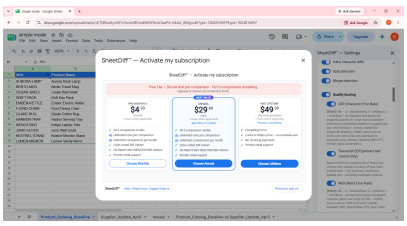
The public listing on the Google Workspace Marketplace.



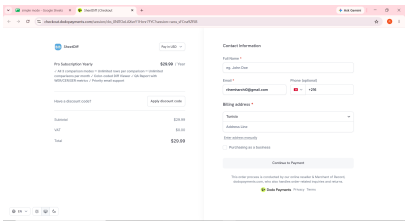
It lives in the Extensions menu, next to the sheet being reviewed.



Choosing a baseline and the versions to compare against it.



Free tier, monthly, annual and lifetime, with the limits stated plainly.



Checkout through a merchant of record, so tax and compliance are handled.

The decisions that shaped it

- **I was the ground truth.** Deciding what happened to a row is hard when speech overlaps: an interruption belongs to the speaker being interrupted and appears again as its own segment for the interrupter, and splitting a segment necessarily rewrites its timestamps. I classified batches of output by hand and sent back every disagreement until the engine matched how a reviewer actually reads a transcript.
- **Grade the work, don't just colour it.** QA exists to tell a project manager how good a transcriptionist was, so the tool measures edit distance and reports it on a scale they can act on.
- **Colour is a hazard.** Marking merges in red risked reviewers reading them as deletions. Legends show only the categories actually present, so the display adds no noise for people working at speed.
- **Say which limit was hit.** Running out of monthly comparisons and exceeding the row cap are different problems and the message has to name the right one.
- **Ten free comparisons**, worked out from how many it takes to try every mode once, rather than picked as a round number.
- **Do not oversell the engine.** Accuracy is relative and probabilistic, and it is presented that way instead of as a headline figure.
- **Four modes, because one comparison does not fit every sheet.** *Structural Diff* tracks rows across two versions and catches the hard cases where a row was split or merged. *Cell-by-Cell* lines the two sheets up and shows exactly which cells changed. *Duplicate Finder* looks inside a single sheet for repeated rows. *Grid Diff* takes any sheet at all with nothing to configure. The settings panel shows only the options belonging to the mode in use, so nobody is asked to reason about a switch that does nothing.

- **One baseline, many sheets, one run.** Reviewers work through a batch, not a single file, so a whole set can be compared against one reference in a single pass with each result written to its own sheet.
- **Tell people what it will cost them before they commit.** The picker shows row counts and an estimated run time up front, and caps the batch so a run cannot die halfway through against the platform's execution limit.
- **Large sheets stay readable.** Output rows are capped and unchanged rows collapse by default, because a diff of ten thousand rows that shows everything is the same problem as no diff at all.
- **Long jobs survive the platform's time limit.** Google stops any single run after six minutes, which is less than a big comparison needs. The work is broken into pieces that each finish inside the limit and carry their state forward, so a large sheet completes across several passes instead of failing at minute six with nothing to show for it.

Designed by the agent: the matching algorithm and its tuning, the licensing and billing lifecycle, and the usage-quota model.

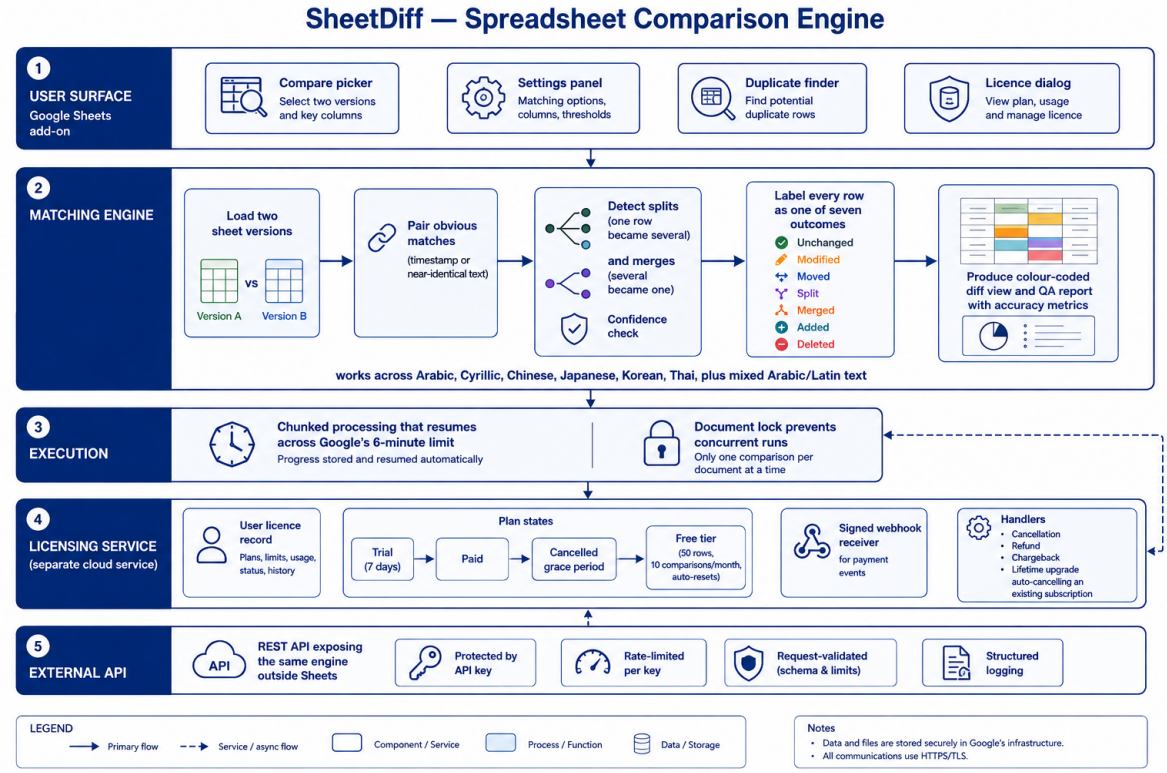


Figure 2: The whole system. The add-on is the surface; underneath sit the matching engine, the execution layer that survives Google's time limit, a separate licensing service, and the same engine exposed as an API for use outside Sheets.

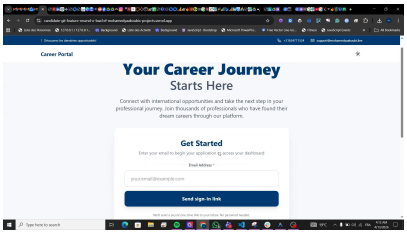
PRODUCT 3

Career Portal

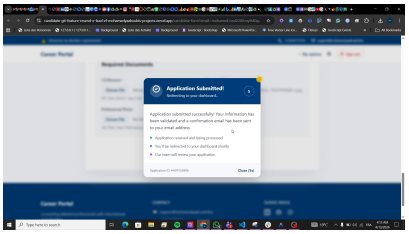
Hiring platform for international placement

A recruitment company places international workers with European employers. Candidates apply with their documents and follow their own status; staff score applications, book interviews that generate their own video link and calendar invite in the employer's timezone, manage listings, and export the pipeline. The client set one hard constraint at the start: candidates will not maintain accounts, so signing in had to work without one.

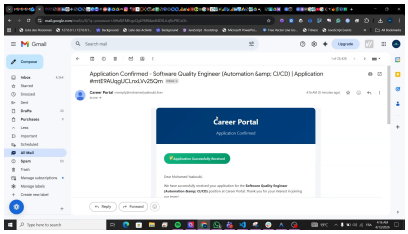
candidate-git-feature-resend-e-baa1ef-mohamedyaakoubis-projects.vercel.app



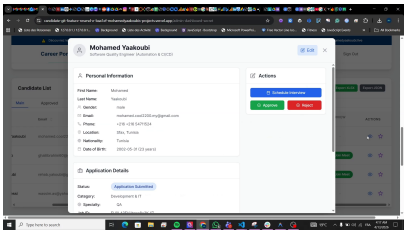
Applicants start with an email address, not a registration form.



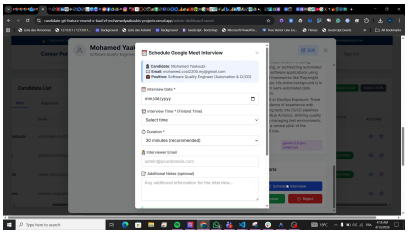
Submission returns an application ID and a confirmed next step.



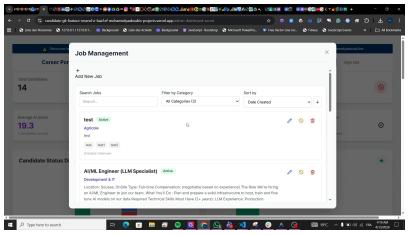
The confirmation arrives by email, carrying the same reference.



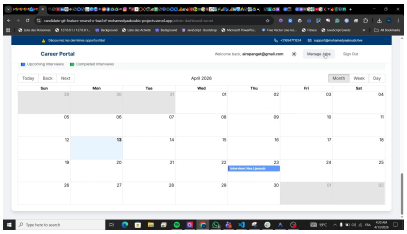
A full candidate record on the staff side, with scoring and decisions in one place.



Interviews are booked in the employer's timezone and generate their own meeting link.



Listings are managed in the product, so the application form always offers real jobs.



Scheduled interviews sit on a shared calendar view rather than in someone's inbox.

The decisions that shaped it

- **An email gate in front of the form.** Applicants enter an address first. Returning applicants are recognised and offered either a fresh application or the ones they already have, which removes duplicate records and stops the form being a soft target for junk.
- **Check the free thing before the paid one.** Verifying an address through a paid service on every attempt burns quota and budget, so the database answers first and the paid check only runs when it must. The same service is throttled to one call a second, because that is what its free allowance permits and exceeding it would fail applications rather than merely cost money.
- **Rules that match how hiring works.** No reapplying to the same role within six months. Applicants must be over eighteen. A returning applicant's name and date of birth are locked while the rest stays editable.
- **A rejected rule.** Blocking applicants whose email address scored poorly would have turned away people

with perfectly ordinary addresses, so it did not ship.

- **Staff can correct a candidate's record**, because someone always calls to say their date of birth is wrong, and a system with no answer to that creates work for everyone.
- **Monitoring that only speaks when it matters.** Alerts fire when activity crosses a threshold that looks like an attack, a burst of failed logins or a flood of requests, rather than on every stray error. An alert channel that cries wolf gets muted, and then it is worth nothing.
- **An upload has to actually be what it claims.** Candidates send passports and CVs, and anyone can rename a file so it ends in `.pdf`. Every upload is opened and checked against the format it says it is, so a file that is something else entirely is refused instead of being stored and later opened by a member of staff.
- **Mail that can be answered.** Interview invitations come from an address a candidate can reply to; rejections do not.

Designed by the agent: the architecture and the security mechanisms, and the diagnosis of the two significant defects found during the build. My part was noticing the symptoms, describing them precisely, and confirming the repairs actually held. One fix was reported as complete when the service it was meant to keep alive had never once been called.

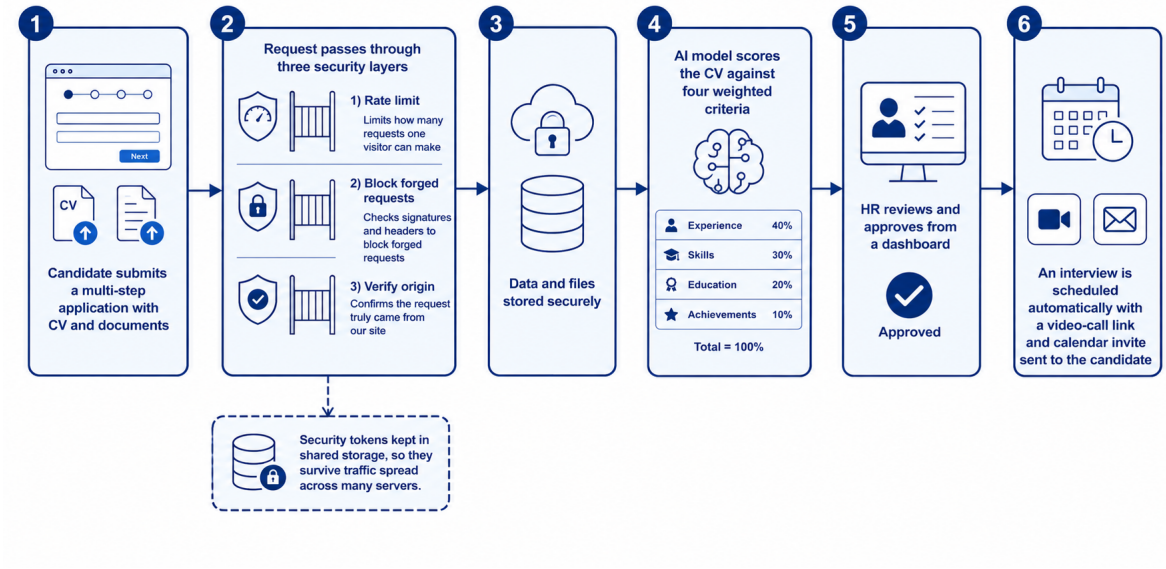


Figure 3: The path an application takes, from submission through to a booked interview.

Shipped with unit, integration and browser-based end-to-end suites across three browser engines, with coverage completed rather than sampled.

Why Me

AI agents have made writing code cheap. They have not made it cheap to know what is worth building, to notice when the answer is wrong, or to care what happens to the person on the other end. That is the work I do, and these three products are the evidence.

I finish things. Not prototypes. A product on a marketplace taking payments, a hiring platform a company runs its pipeline through, and a life-safety app in final testing. Each went from an idea to something people use, including the parts nobody enjoys: taking money and giving it back, moderation queues, letting a user erase themselves properly, timezones, and alerts that fail to arrive.

I bring what the model cannot reach. An agent can build a notification system. It cannot tell you which phones your users actually carry, which of your words will be misread by someone working too fast, or how a customer's ordinary habits will quietly break your billing. That knowledge decides whether a product works, and it has to come from someone who understands the people it is for.

I do not take output on trust. Code that runs and code that is correct are different things, and the gap only appears under real conditions. I test on the hardware people actually own, in the state they will actually be in, and I confirm a fix rather than accept the claim that it is done. That habit is the difference between shipping AI-written code and shipping code that works.

I design for the conditions the product will really meet: no signal, cheap hardware, free tiers, platform limits, and people working too fast to read carefully. Constraints are where most of my decisions come from, and they are usually the ones that decide whether something survives contact with users.

I know where my knowledge stops. I do not write the implementation and I will not claim to debug a stack trace unaided. Saying so is not modesty, it is the thing that makes me safe to hand work to: I escalate early, I do not bluff, and I check before I call something done.

Put me on a product and I will tell you what to build, direct the agents that build it, and be the person who notices before your users do.

Each of these products does considerably more than these pages show. I have kept to the decisions that explain how something was built and why it holds up, on the assumption that your time is short. I am glad to walk through any of it in full.